

MAUI-APP ONTWERPVERANTWOORDING

25/05/2026

1. Opsplitsen van verantwoordelijkheden

Ik heb de applicatie opgebouwd volgens de 3-lagenarchitectuur. De verschillende lagen zijn de data-laag, business-laag en UI-laag.

In de data-laag staat de code specifiek voor LiteDB. Hier staat de connectionklasse en twee repositories. Deze repositories hebben elk een interface in de business-laag. Door het op deze manier te doen zou je nog een ander soort databank kunnen gebruiken zonder iets aan de business-laag of UI-laag te veranderen.

Dan hebben we de business-laag. Hier zitten de modellen Task en Person, de repository-interfaces, de services en de strategieklassen. Hier doe ik alle bedrijfslogica en zorg ik ervoor dat acties verwerkt worden en doorgestuurd worden van de UI naar de data-laag.

Vervolgens hebben we de UI-laag. Deze bestaat uit pagina's en ViewModels. Ik heb de code-behinds zo leeg mogelijk gehouden. De ViewModels zijn ingesteld als BindingContext. Op die manier volg ik het MVVM-patroon zoals we dat in de les gezien hebben.

2. Domeinmodellering

Task en Person zijn eenvoudige klassen zonder gedrag in. Ik heb dit bewust gekozen om ze simpel te houden. Ze bevatten data en geen methodes. De methodes die ze moeten gebruiken staan in de services. Een Task heeft een PersonId. Zo kunnen Task en Person gelinked worden met elkaar zonder dat Task een heel Person instantie moet opslaan. Dit is beter voor de databank en voorkomt dat een Person 2 keer wordt opgeslagen.

Beide klassen zijn voorzien van een CreatedAt en een UpdatedAt veld. Deze worden ingevuld of geupdate bij het aanmaken of bewerken van een Task of Person. Door deze info kunnen we gemakkelijk filteren met het strategie-patroon.

3. Navigatie en communicatie

De navigatie verloopt via een NavigationService. Dit is een wrapper rond Shell.Current, zodat ViewModels nooit rechtstreeks van MAUI-klassen afhangen. Parameters worden meegegeven via ShellNavigationQueryParameters. De ViewModel die het ontvangt gebruikt IQueryAttributable. Wanneer een taak of persoon opgeslagen of verwijderd wordt, stuurt de service een bericht via WeakReferenceMessenger. De lijstschermpjes passen enkel het betrokken item aan in de ObservableCollection zonder de volledige lijst opnieuw op te halen.

4. Toegepaste patronen

MVVM: Ik gebruik MVVM omdat schermen dan geen businesslogica bevatten. De base class ViewModel implementeert INotifyPropertyChanged, zodat wijzigingen automatisch zichtbaar worden in de UI. Acties zoals opslaan, annuleren, openen, nieuw maken, filteren en checkbox-updates gebeuren via ICommand. TaskViewModel gebruikt een callback wanneer IsCompleted verandert, zodat TaskService.ToggleCompleted de wijziging meteen kan opslaan.

DI / IoC: Alles wordt via de constructor geïnjecteerd. Registraties in MauiProgram.cs zijn opgesplitst per categorie. Afhankelijkheden worden via constructors binnengebracht en hierdoor maak ik niet overal zelf objecten aan met new.

Strategy: Filteren en sorteren. ITaskListStrategy definieert Name, RequiresReorder en Apply(tasks). Er zijn drie strategieën: AllTasksStrategy, UnfinishedTasksStrategy en MostRecentlyUpdatedTasksStrategy. De TaskListModel gebruikt CurrentStrategy.RequiresReorder om te beslissen of een volledige herlaad nodig is zonder een hardcoded type-check. Een nieuwe strategie toevoegen vereist enkel een nieuwe klasse; bestaande code blijft onaangeroerd (Open/Closed principe).

Messaging/Observer: Services sturen berichten wanneer een taak of persoon wordt opgeslagen of verwijderd. De ListViewModels luisteren naar die berichten en passen hun ObservableCollection aan. Daardoor hoeft de detailpagina de lijstpagina niet rechtstreeks te kennen.

5. Bewuste keuzes

Ik heb niet elk patroon geforceerd. Factory gebruik ik niet, omdat Task en Person eenvoudig aangemaakt worden en geen complexe creatielogica nodig hebben. State Pattern gebruik ik niet, omdat IsCompleted gewoon een boolean is en er geen uitgebreide toestanden met overgangslogica zijn.

Conclusie

Mijn ontwerp verdeelt verantwoordelijkheden duidelijk. Pages tonen de UI, ViewModels sturen de schermlogica, services bevatten regels en validatie, repositories schermen LiteDB af, strategies maken filtergedrag uitbreidbaar en messages zorgen voor losse communicatie tussen schermen. Daardoor blijft mijn TodoApp leesbaar, onderhoudbaar en uitbreidbaar zonder onnodige patronen te forceren.